



Федеральное государственное автономное образовательное

учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное
обеспечение

Дисциплина «Вычислительная математика»

Отчёт по лабораторной работе №4

Вариант - 6

Выполнил

Линейский Аким Евгеньевич

P3215

Проверил

Наумова Надежда Александровна

Содержание

1. Содержание.....	2
2. Цель	3
3. Порядок выполнения работы.....	3
4. Функция по заданию.....	4
5. Линейное приближение.....	5
6. Квадратичное приближение	6
7. Анализ приближений.....	7
8. Вычислительная часть	8
Интерфейс программы	8
9. Блок схемы численных методов.....	9
10. Исходный программы.....	10
11. Вывод	13

Цель

Найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов

Порядок выполнения работы

1. Вычислительная реализация задачи

- 1.1. Сформировать таблицу табулирования заданной функции на указанном интервале
- 1.2. Построить линейное и квадратичное приближения по 11 точкам заданного интервала
- 1.3. Найти среднеквадратические отклонения для каждой аппроксимирующей функции
- 1.4. Выбрать наилучшее приближение
- 1.5. Построить графики заданной функции, а также полученные линейное и квадратичное приближения

2. Программная реализация задачи

2.1. Программа

- 2.1.1. Реализовать метод наименьших квадратов, исследуя все указанные функции
- 2.1.2. Предусмотреть ввод исходных данных из файла/консоли, вывод результатов в файл/консоль: коэффициенты аппроксимирующих функций, среднеквадратичное отклонение, массивы значений
- 2.1.3. Для линейной зависимости вычислить коэффициент корреляции Пирсона
- 2.1.4. Организовать вывод графиков функций, графики должны полностью отображать весь исследуемый интервал
- 2.1.5. Программа должна быть протестирована при различных наборах данных, в том числе и некорректных

2.2. Методика проведения исследования

- 2.2.1. Вычислить меру отклонения исследуемых функций

- 2.2.2. Уточнить значения коэффициентов эмпирических функций, минимизируя функцию S;
- 2.2.3. Сформировать массивы предполагаемых эмпирических зависимостей
- 2.2.4. Определить среднеквадратичное отклонение для каждой аппроксимирующей функции. Выбрать наименьшее значение и, следовательно, наилучшее приближение
- 2.2.5. Построить графики полученных эмпирических функций

Функция по заданию

$$y = \frac{12x}{x^4 + 6}, \quad x \in [0, 2] \quad h = 0,2$$

X	Y
0.0	0.0
0.2	0.399893
0.4	0.796601
0.6	1.174628
0.8	1.497753
1.0	1.714285
1.2	1.783590
1.4	1.707039
1.6	1.529441
1.8	1.309281
2.0	1.090909

Табл 1. Табулирование функции на интервале.

Линейное приближение

$$\begin{cases} a \sum_{i=1}^n x_i^2 + b \sum_{i=1}^n x_i = \sum_{i=1}^n x_i y_i \\ a \sum_{i=1}^n x_i + b n = \sum_{i=1}^n y_i \end{cases}$$

$$SX = \sum_{i=1}^n x_i = 11$$

$$SXX = \sum_{i=1}^n x_i^2 = 15.4$$

$$SY = \sum_{i=1}^n y_i = 13.003$$

$$SXY = \sum_{i=1}^n x_i y_i = 15.532$$

$$\Delta = SXX \cdot n - SX \cdot SX = 48.4$$

$$\Delta_1 = SXY \cdot n - SX \cdot SY = 27.811$$

$$\Delta_2 = SXX \cdot SY - SX \cdot SXY = 29.404$$

$$a = \frac{\Delta_1}{\Delta} = 0.575, \quad b = \frac{\Delta_2}{\Delta} = 0.607$$

$$P_1(x) = 0.575x + 0.607$$

№	X	Y	P1(x)	E _i
1	0.0	0.000	0.608	0.608
2	0.2	0.400	0.722	0.323
3	0.4	0.797	0.839	0.041
4	0.6	1.175	0.952	-0.222
5	0.8	1.498	1.067	-0.431
6	1.0	1.714	1.182	-0.532
7	1.2	1.784	1.297	-0.487

8	1.4	1.707	1.412	-0.295
9	1.6	1.529	1.527	-0.003
10	1.8	1.309	1.642	0.333
11	2.0	1.091	1.757	0.666

$$\delta_1 = \sqrt{\frac{\sum_{i=1}^n (\varphi(x_i) - y_i)^2}{n}} = 0.412$$

Квадратичное приближение

$$\varphi(x, a_0, a_1, a_2) = a_0 + a_1 x + a_2 x^2$$

$$S = S(a_0, a_1, a_2) = \sum_{i=1}^n (a_0 + a_1 x_i + a_2 x_i^2 - y_i)^2 \rightarrow \min$$

$$\left\{ \begin{array}{l} a_0 n + a_1 \sum_{i=1}^n x_i + a_2 \sum_{i=1}^n x_i^2 = \sum_{i=1}^n y_i \\ a_0 \sum_{i=1}^n x_i + a_1 \sum_{i=1}^n x_i^2 + a_2 \sum_{i=1}^n x_i^3 = \sum_{i=1}^n x_i y_i \\ a_0 \sum_{i=1}^n x_i^2 + a_1 \sum_{i=1}^n x_i^3 + a_2 \sum_{i=1}^n x_i^4 = \sum_{i=1}^n x_i^2 y_i \end{array} \right.$$

$$\left\{ \begin{array}{l} 11a_0 + 11a_1 + 15.4a_2 = 13.003 \\ 11a_0 + 15.4a_1 + 24.2a_3 = 15.532 \\ 15.4a_0 + 24.2a_1 + 40.532a_2 = 21.674 \end{array} \right.$$

$$a_0 = -0.087, \quad a_1 = 2.889, \quad a_2 = -1.158$$

$$P_2(x) = -1.158x_i^2 + 2.889x_i - 0.087$$

№	X	Y	P2(x)	E_i
1	0.0	0.000	-0.087	-0.087
2	0.2	0.400	0.444	0.045

3	0.4	0.797	0.883	0.087
4	0.6	1.175	1.230	0.055
5	0.8	1.498	1.483	-0.015
6	1.0	1.714	1.644	-0.070
7	1.2	1.784	1.712	-0.071
8	1.4	1.707	1.688	-0.019
9	1.6	1.529	1.571	0.041
10	1.8	1.309	1.361	0.052
11	2.0	1.091	1.059	-0.032

$$\delta_2 = \sqrt{\frac{\sum_{i=1}^n (\varphi(x_i) - y_i)^2}{n}} = 0.057$$

Анализ приближений

Среднеквадратическое отклонение линейной аппроксимации:

$$\delta_1 = 0.412$$

Среднеквадратическое отклонение квадратичной аппроксимации:

$$\delta_2 = 0.057$$

Лучшее приближение показывает квадратичная аппроксимация.

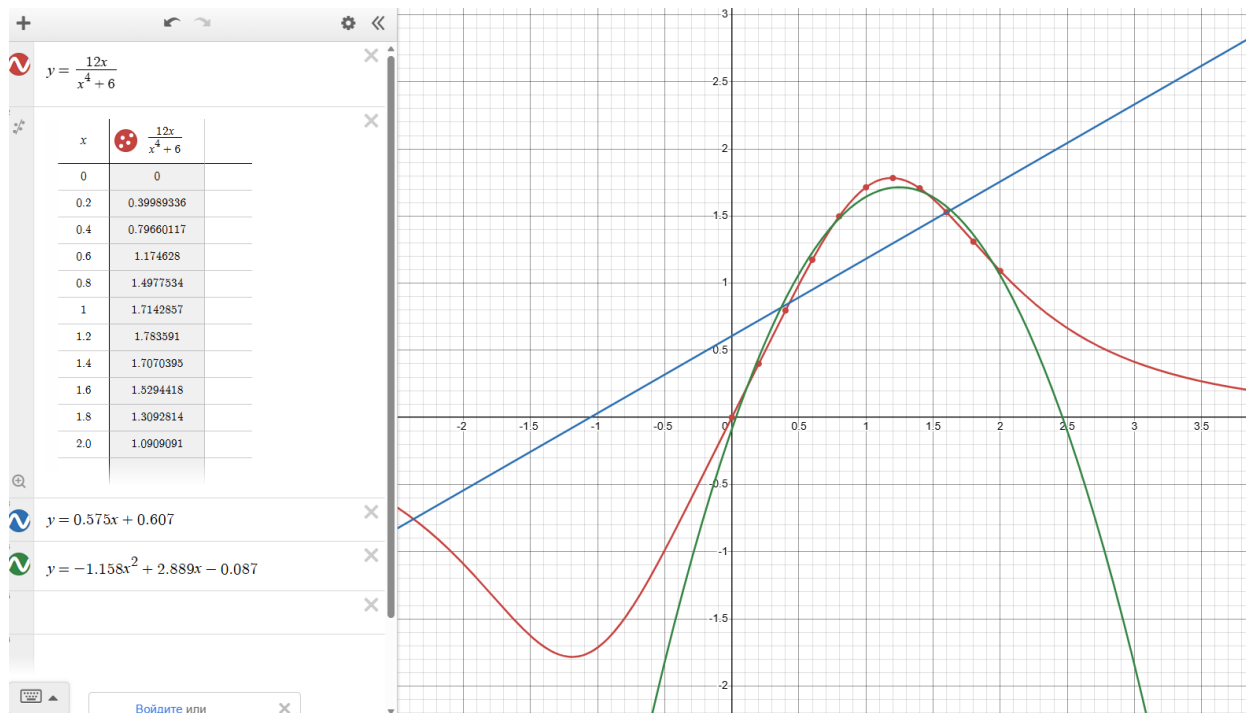
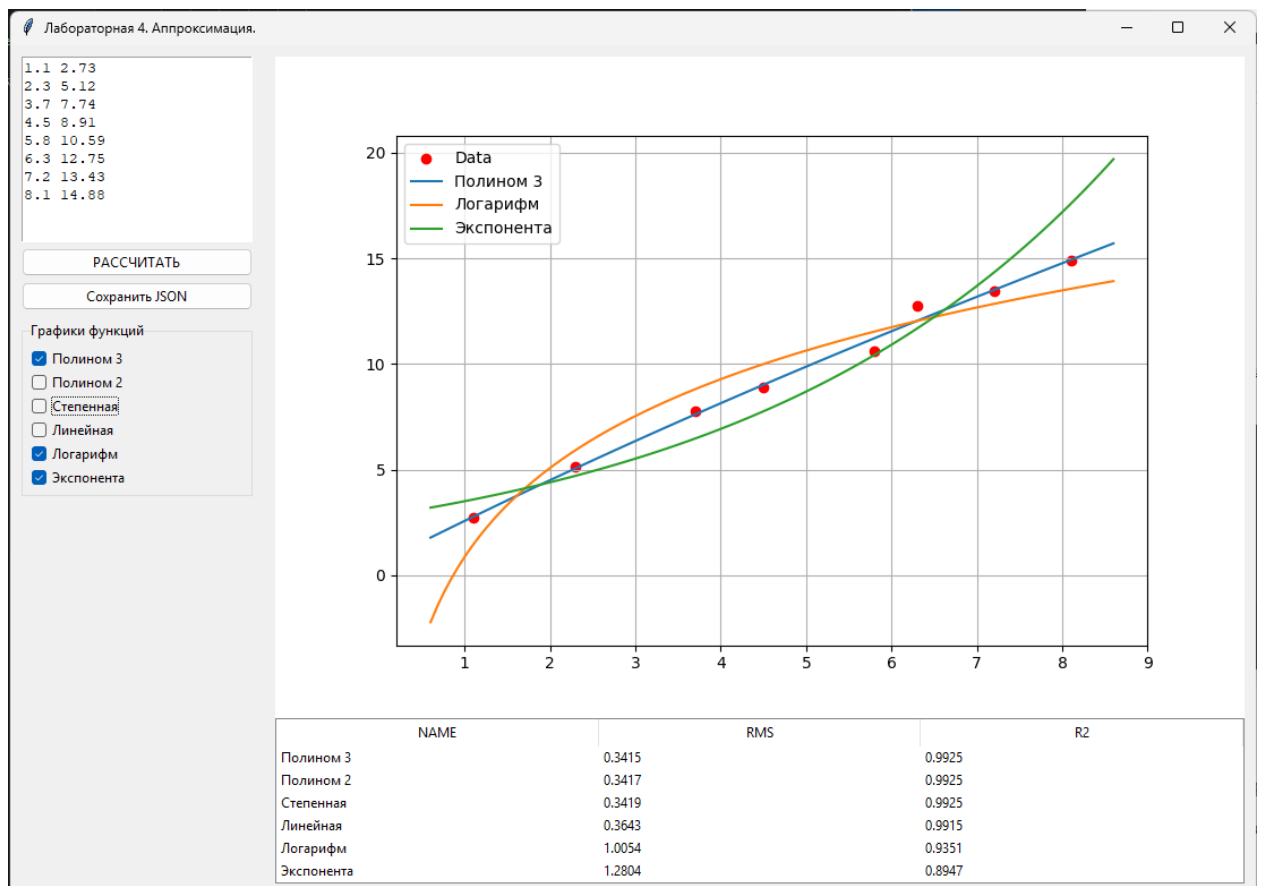


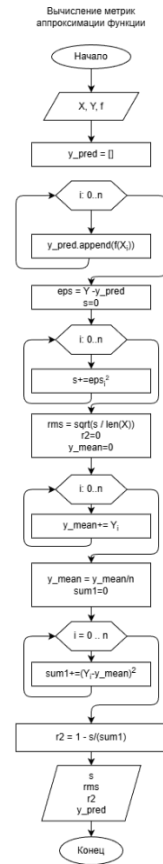
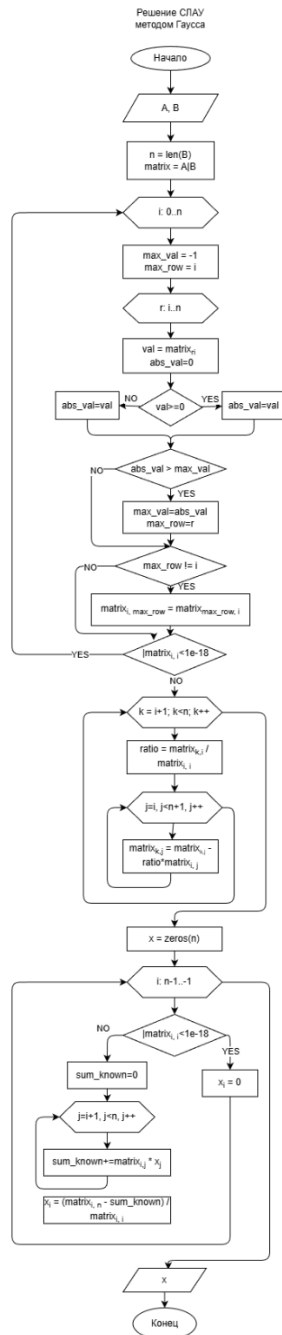
График функций

Вычислительная часть

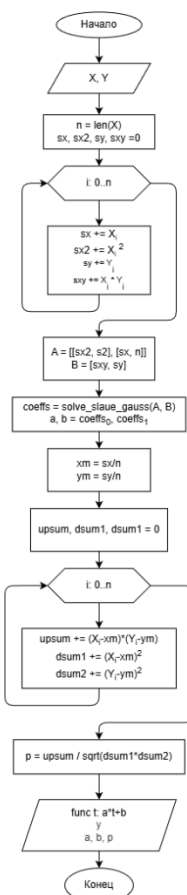
Интерфейс программы



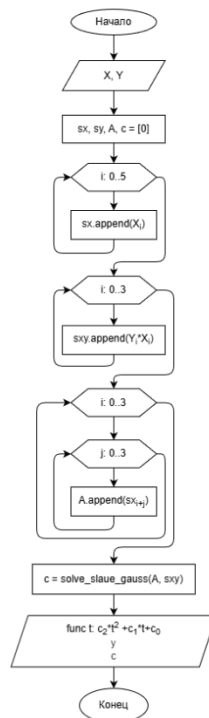
Блок схемы численных методов



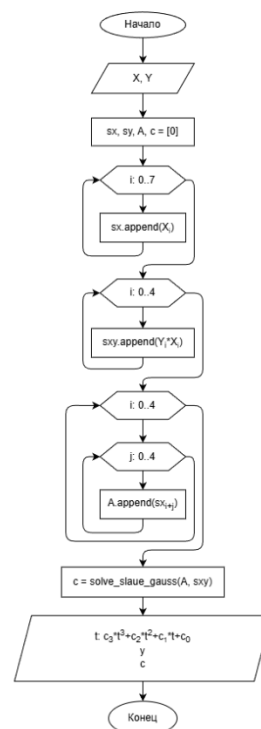
Линейная аппроксимация



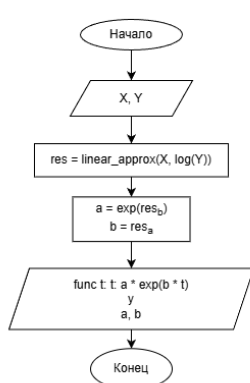
Полином 2 степени



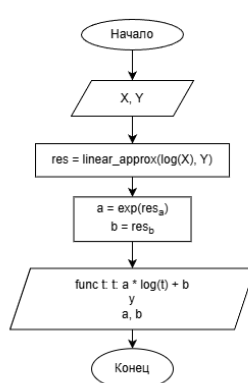
Полином 3 степени



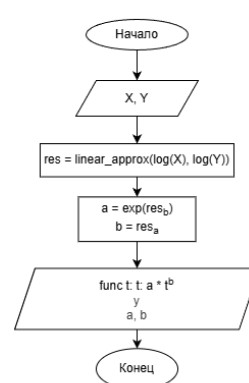
Экспоненциальная функция



Логарифмическая функция



Степенная функция



Исходный программы

```
class MathCore:
    @staticmethod
```

```

def solve_slaue_gauss(A, B):
    """метод Гаусса с выбором главного элемента"""
    try:
        n = len(B)
        matrix = np.column_stack((A, B)).astype(np.float64)
        for i in range(n):
            max_row = i + np.argmax(np.abs(matrix[i:, i]))
            matrix[[i, max_row]] = matrix[[max_row, i]]
            if abs(matrix[i, i]) < 1e-18:
                continue
            for j in range(i + 1, n):
                ratio = matrix[j, i] / matrix[i, i]
                matrix[j, i:] -= ratio * matrix[i, i:]
        x = np.zeros(n)
        for i in range(n - 1, -1, -1):
            if abs(matrix[i, i]) < 1e-18:
                x[i] = 0
            else:
                x[i] = (
                    matrix[i, n] - np.dot(matrix[i, i + 1 : n], x[i + 1 : n])
                ) / matrix[i, i]
        return x
    except:
        return np.zeros(len(B))

@staticmethod
def calculate_metrics(x, y, f):
    y_pred = np.array([f(val) for val in x])
    eps = y - y_pred
    s = np.sum(eps**2)
    rms = np.sqrt(s / len(x))
    r2 = 1 - (s / np.sum((y - np.mean(y)) ** 2)) # Коэффициент детерминации
    return {
        "S": float(s),
        "RMS": float(rms),
        "R2": float(r2),
        "y_pred": y_pred.tolist(),
    }

@staticmethod
def linear_approx(x, y):
    n = len(x)
    sx, sy, sx2, sxy = np.sum(x), np.sum(y), np.sum(x**2), np.sum(x * y)
    A = [[sx2, sx], [sx, n]]
    B = [sxy, sy]
    coeffs = MathCore.solve_slaue_gauss(A, B) # [a, b]
    a, b = coeffs[0], coeffs[1]

    xm, ym = np.mean(x), np.mean(y)
    p = np.sum((x - xm) * (y - ym)) / (
        np.sqrt(np.sum((x - xm) ** 2) * np.sum((y - ym) ** 2)) or 1
    )
    return (

```

```

        (lambda t: a * t + b),
        f'y = {a:.4f}x + {b:.4f}",
        {"a": a, "b": b, "P": p},
    )

    @staticmethod
    def poly2_approx(x, y):
        sx = [np.sum(x**i) for i in range(5)]
        sxy = [np.sum(y * (x**i)) for i in range(3)]
        A = [[sx[i + j] for j in range(3)] for i in range(3)]
        c = MathCore.solve_slaue_gauss(A, sxy) # [a0, a1, a2]
        return (
            (lambda t: c[2] * t**2 + c[1] * t + c[0]),
            f'y = {c[2]:.4f}x^2 + {c[1]:.4f}x + {c[0]:.4f}",
            c.tolist(),
        )

    @staticmethod
    def poly3_approx(x, y):
        sx = [np.sum(x**i) for i in range(7)]
        sxy = [np.sum(y * (x**i)) for i in range(4)]
        A = [[sx[i + j] for j in range(4)] for i in range(4)]
        c = MathCore.solve_slaue_gauss(A, sxy)
        return (
            (lambda t: c[3] * t**3 + c[2] * t**2 + c[1] * t + c[0]),
            f'y = {c[3]:.4f}x^3 + {c[2]:.4f}x^2 + {c[1]:.4f}x + {c[0]:.4f}",
            c.tolist(),
        )

    @staticmethod
    def exp_approx(x, y):
        if np.any(y <= 0):
            return None, None, None
        f_lin, _, res = MathCore.linear_approx(x, np.log(y))
        a, b = np.exp(res["b"]), res["a"]
        return (
            (lambda t: a * np.exp(b * t)),
            f'y = {a:.4f} * e^{(b:.4f)x}',
            {"a": a, "b": b},
        )

    @staticmethod
    def log_approx(x, y):
        if np.any(x <= 0):
            return None, None, None
        f_lin, _, res = MathCore.linear_approx(np.log(x), y)
        return (
            (lambda t: res["a"] * np.log(t) + res["b"]),
            f'y = {res["a"]:.4f}ln(x) + {res["b"]:.4f}',
            res,
        )

    @staticmethod

```

```
def pow_approx(x, y):
    if np.any(x <= 0) or np.any(y <= 0):
        return None, None, None
    f_lin, _, res = MathCore.linear_approx(np.log(x), np.log(y))
    a, b = np.exp(res["b"]), res["a"]
    return (lambda t: a * (t**b)), f"y = {a:.4f} * x^{b:.4f}", {"a": a, "b": b}
```

Код доступен по ссылке на удаленный репозиторий GitHub:

[Код программы](#)

Вывод

Проделав данную лабораторную работу №4, я изучил различные способы аппроксимации функций, на практике исследовал точность приближения различных типов функций, реализовал численные методы и графический интерфейс с просмотром графиков и сохранением отчета вычислений на языке Python.

